ASSIGNMENT #01

SEED Lab1

NAME: MUHAMMAD BILAL

ROLL NO:, 22I-1533

DEGREE PROGRAM: BS SOFTWARE ENGINEERING (BATCH 2022)

SECTION: G

SUBMITTED TO: Sir Zaheer Sani Sahb

Information Security

FAST-NUCES (Islamabad Campus)

SUBMISSION DATE: 28 SEP 2025

Table of Contents

Purpose of the Lab: 3
Environment setup — VM and Docker (step-by-step): 3
Download & import SEED VM 4
Set wallpaper (so it appears on screenshots) 4
Install Docker Inside SEED Ubuntu: 5
Building of Docker Contianer and Server Running Evidence: 6
Authentication bypass : 7
Extracting data using UNION or Boolean-based techniques: 8
Modification of Data Or Updating of data via SQL Query: 11
Demonstration of prevention: 12
Prevention Proof: 12
Steps: 13
What I learned from Lab 1 (Reflection): 15

Task 1

Purpose of the Lab:

The purpose of the Web SQL Injection lab is to learn how SQL Injection (SQLi) vulnerabilities appear in web applications, how attackers exploit them to read or modify data, and which defensive measures prevent these exploits. Specifically, the lab demonstrates:

- How unsafely-constructed SQL queries (string concatenation with user input) allow authentication bypass, data extraction, and data modification.
- Several SQLi techniques (authentication bypass, boolean-based retrieval, UNION-based extraction).
- How to fix vulnerable code using parameterized queries, input validation/escaping, least-privilege database accounts, and secure configuration.

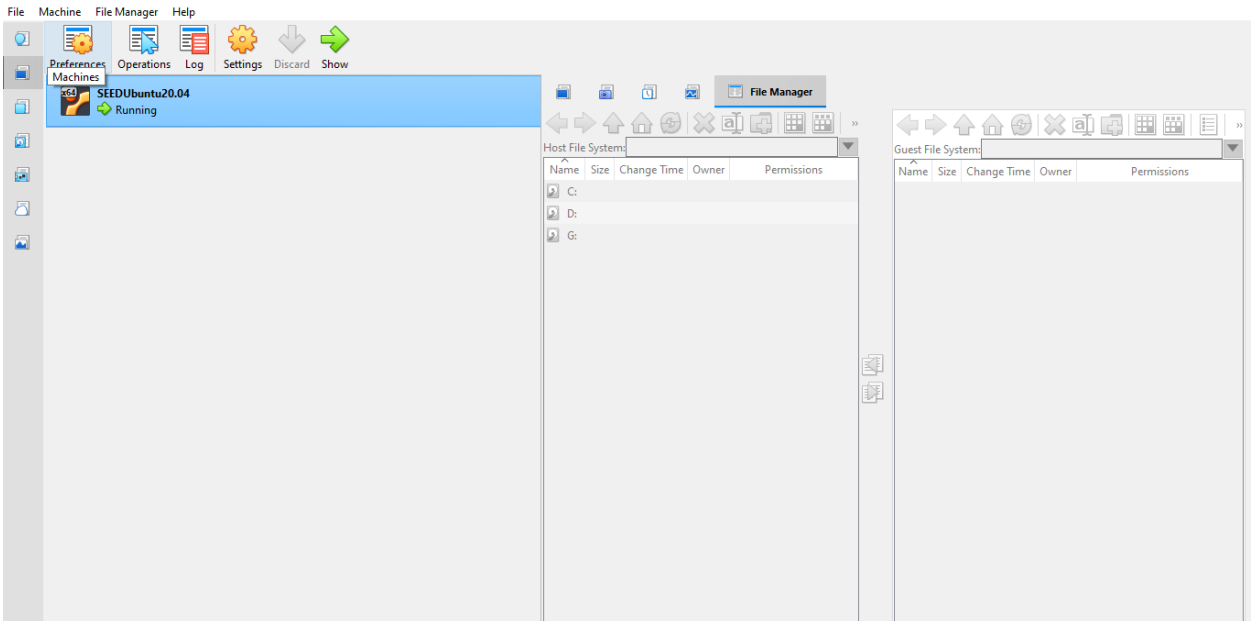
Environment setup — VM and Docker (step-by-step):

Note: I used the official SEED Ubuntu 20.04 VM (imported into VirtualBox / VMware Fusion). The VM must show a wallpaper with my **name** and **REG#** in screenshots (add your name &

regno and set as desktop wallpaper before screenshots). Below are the steps I followed from downloading the VM to running the vulnerable web app with Docker inside the VM.

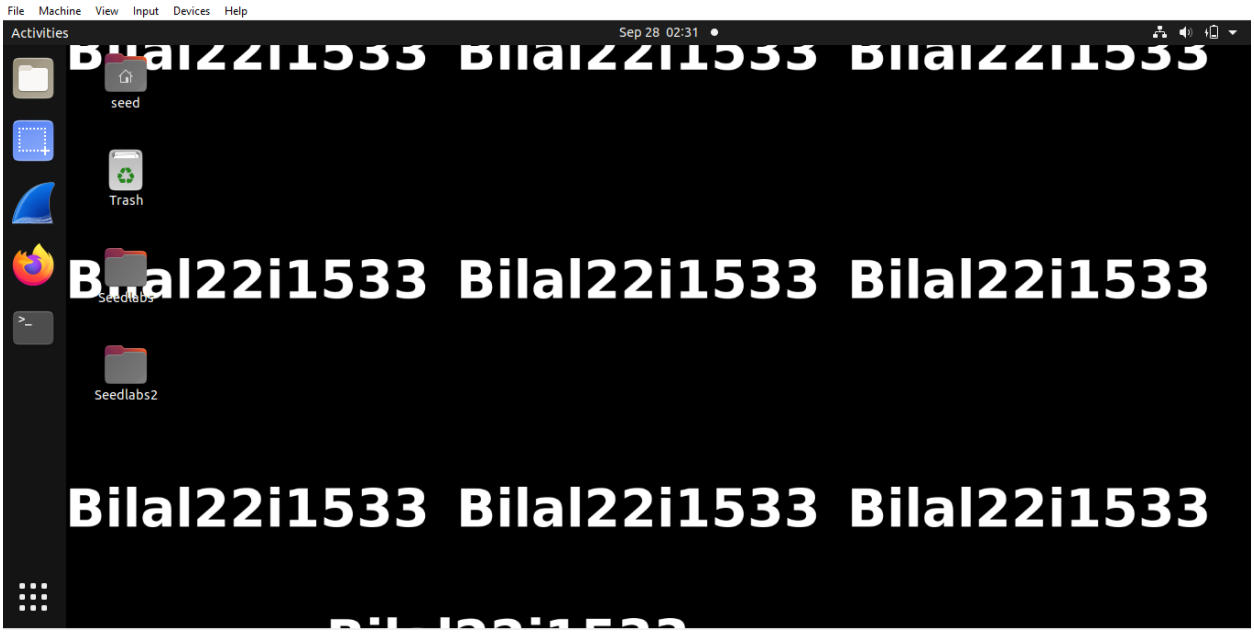
Download & import SEED VM

1. Download the SEED VM image (Ubuntu 20.04) from the official SEED Labs release page (the file is ~4GB).
2. Open VirtualBox or VMware and create a new VM:
    - Type: Linux → Ubuntu (64-bit)
    - Memory: at least 2048 MB (4096 MB recommended)
    - Storage: attach the downloaded SEED VM disk (or import the provided .ova)
3. Boot the VM and log in with the default SEED credentials (follow the lab instructions for username/password).



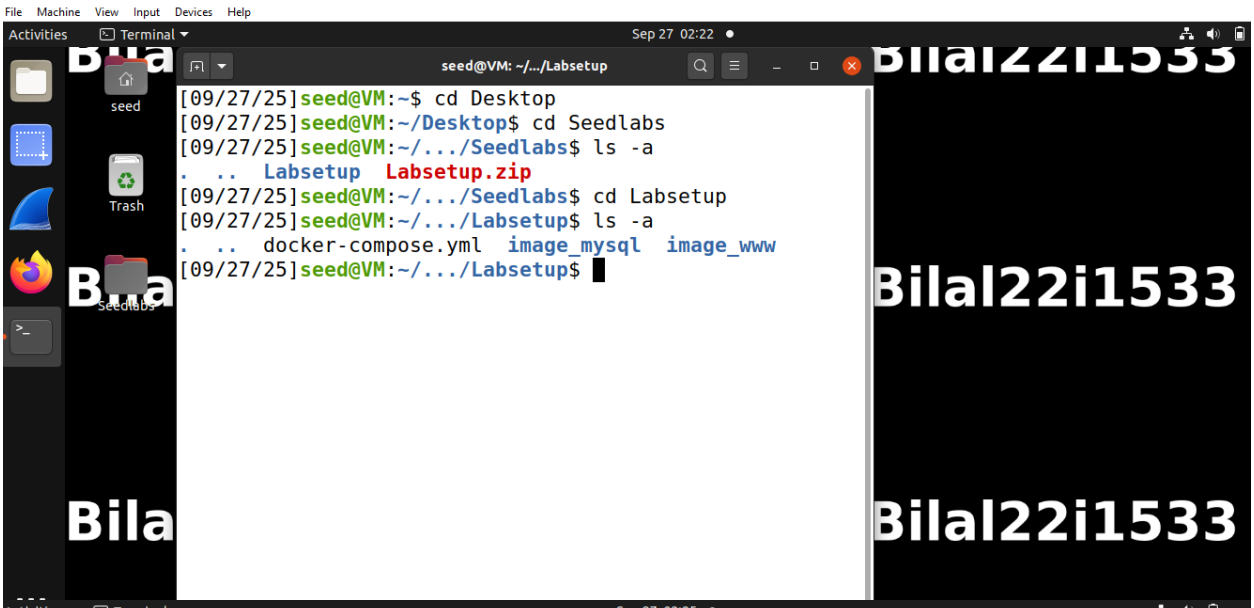
Set wallpaper (so it appears on screenshots)

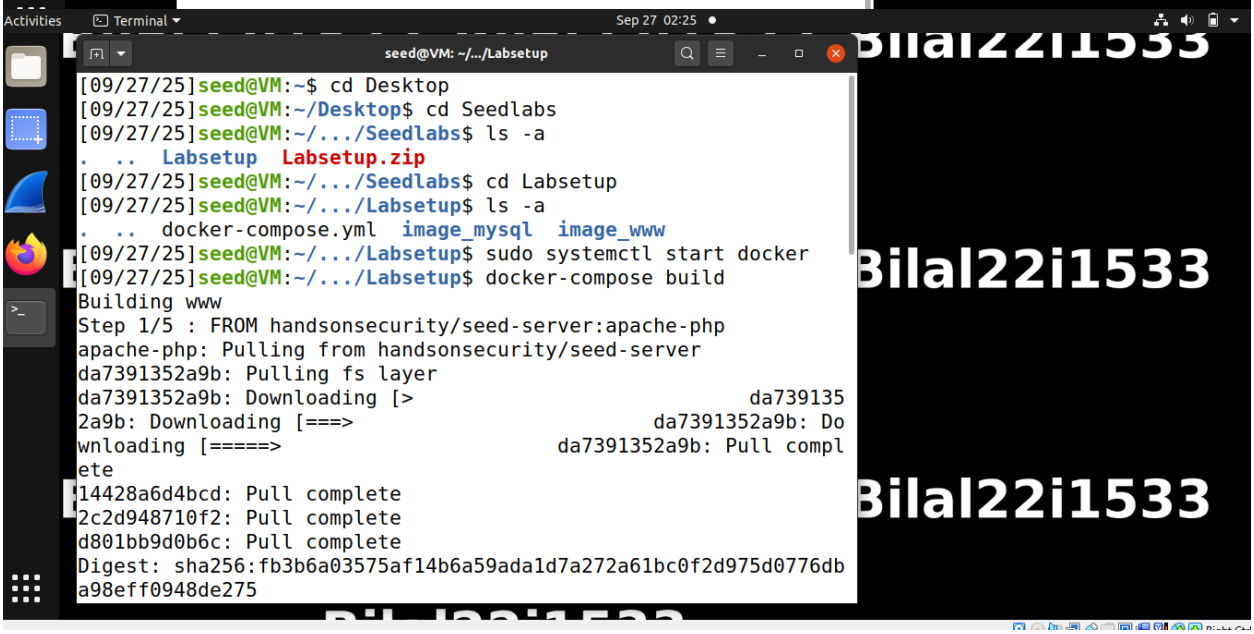
1. Create an image file containing your name and regno (e.g., using GIMP or `convert` from ImageMagick).



Install Docker Inside SEED Ubuntu:

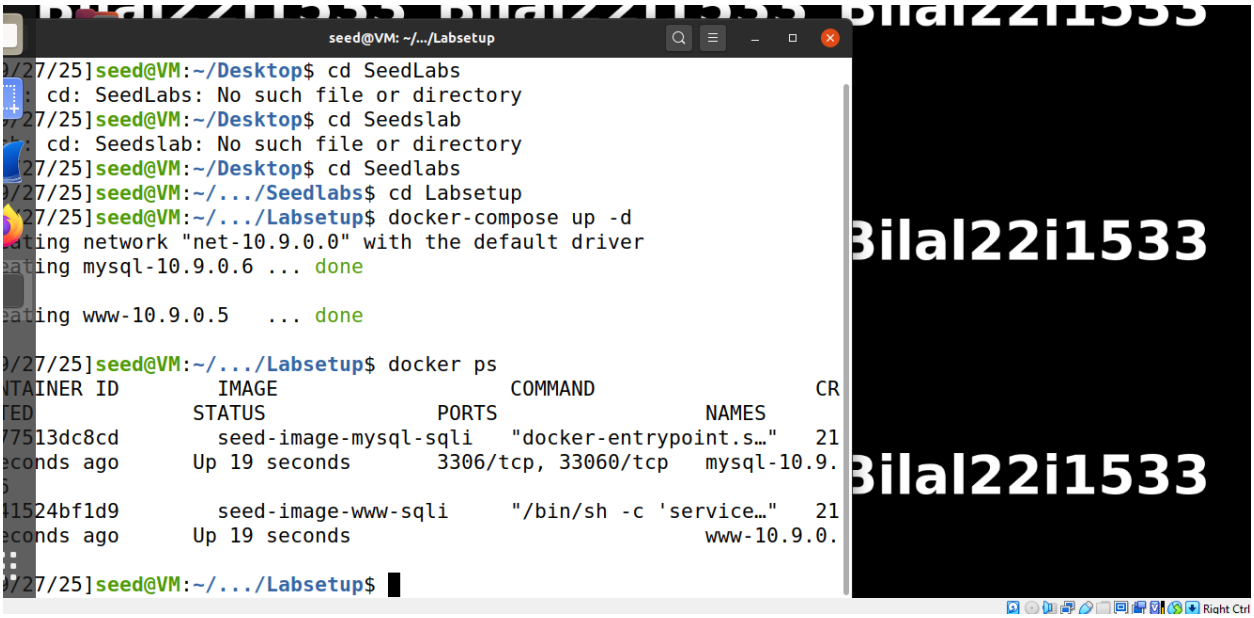
Successfully Installed Docker and Docker-compose for Lab Work

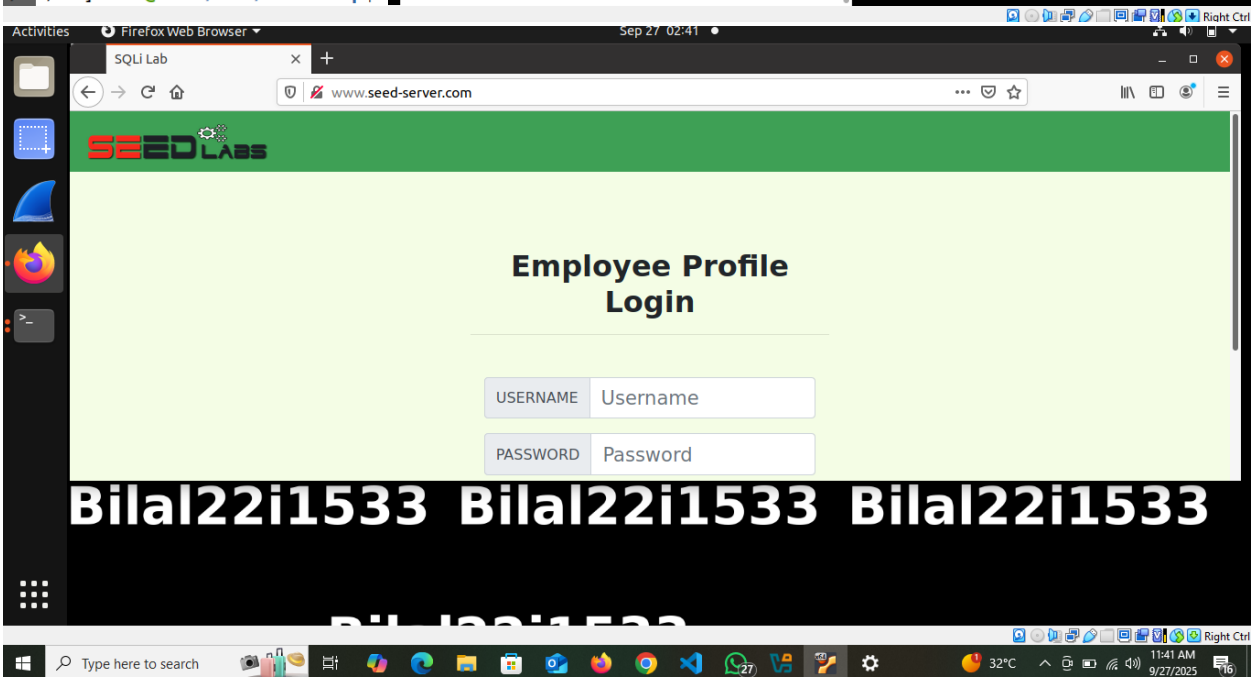




Building of Docker Contianer and Server Running Evidence:

I have successfully run my application in the browser for performs Required Task:



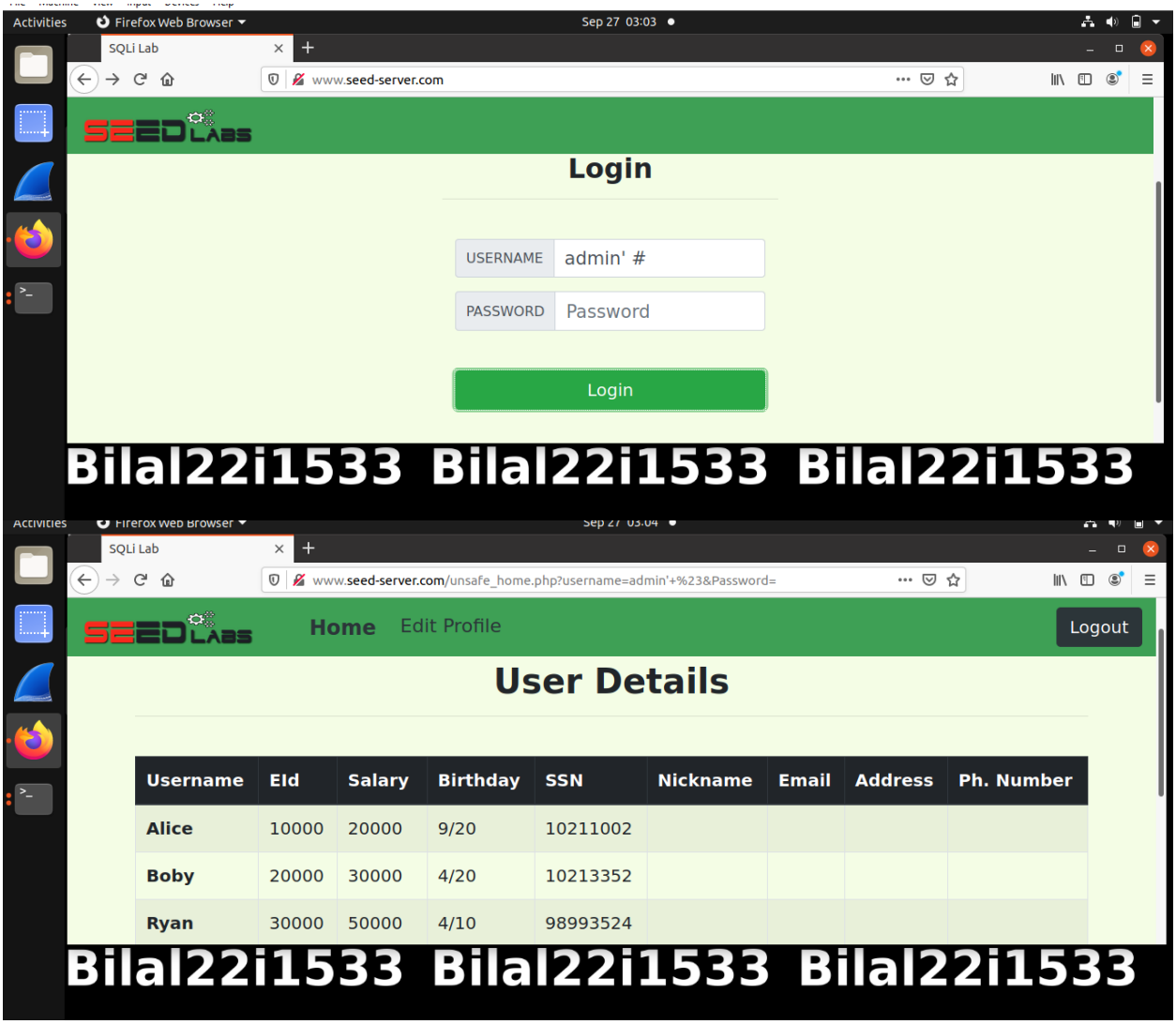


Authentication bypass :

On a login form that uses SQL to validate user credentials, try the classic payload to bypass authentication:

- Username: `admin'#`
- Password: Blank

I have successfully Login using these credentials which is showing authentication bypass See the Evidence Below:

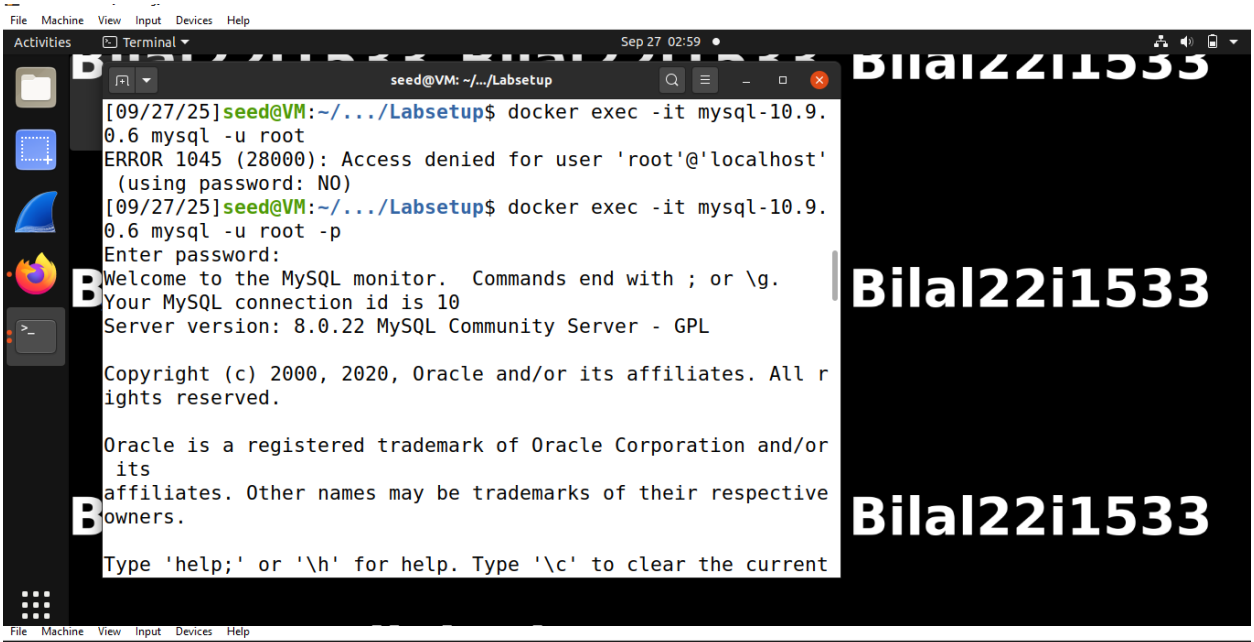


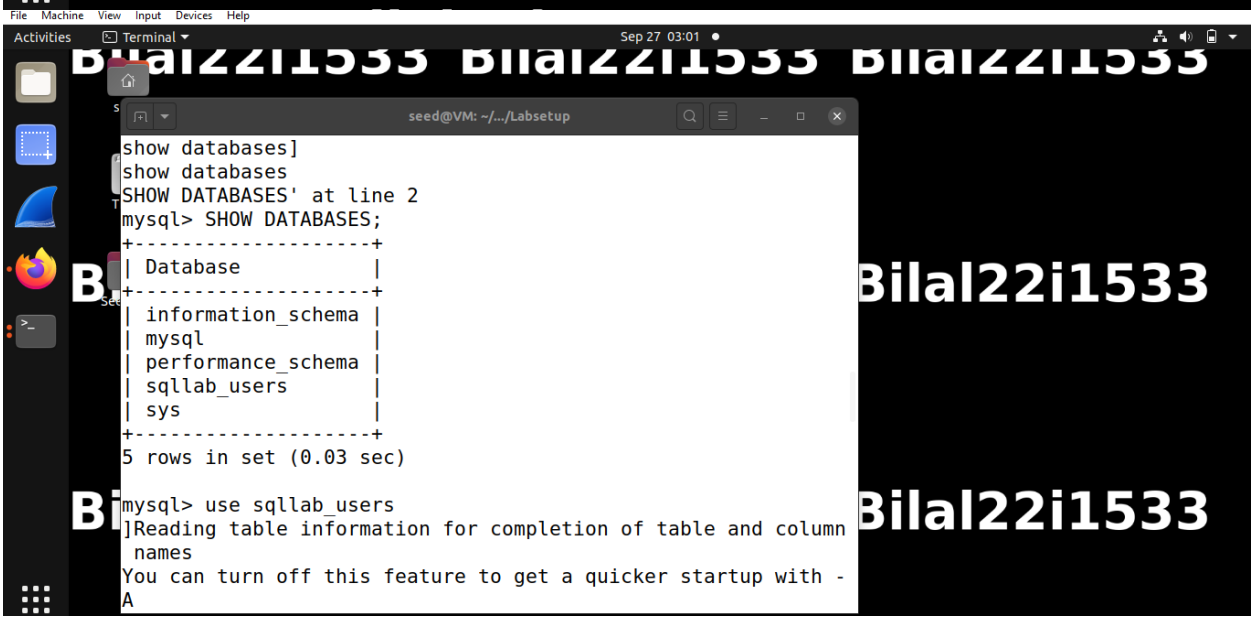
Extracting data using UNION or Boolean-based techniques:

- First of all I have selected Databases.
- Second, I opened the database used by the web application.
- Third, I went to Tables and selected the table named credentials.
- Fourth, I selected all columns from credentials (showing 20 rows the table view limited to 20).
- Fifth, I filtered the table to display only the row where Name = admin.

• Finally, I took a full-screen screenshot showing the result (VM wallpaper with my name & regno, username, date and time visible).

See all Screenshots Below:





```
1 row in set (0.00 sec)

mysql> select * from credential limit 20;
+----+-------+-------+--------+-------+----------+-----------
--+---------+-------+----------+----------------------------
-------------+
| ID | Name  | EID   | Salary | birth | SSN      | PhoneNumbe
r | Address | Email | NickName | Password
             |
+----+-------+-------+--------+-------+----------+-----------
--+---------+-------+----------+----------------------------
-------------+
|  1 | Alice | 10000 |  20000 | 9/20  | 10211002 |
  |         |       |        | fdbe918bdae83000aa54747fc95f
e0470fff4976 |
|  2 | Boby  | 20000 |  30000 | 4/20  | 10213352 |
  |         |       |        | b78ed97677c161c1c82c14290667
4ad15242b2d4 |
|  3 | Ryan  | 30000 |  50000 | 4/10  | 98993524 |
  |         |       |        | a3c50276cb120637cca669eb38fb
```

```
mysql> select * from credential where username = 'admin' limi
t 10;
ERROR 1054 (42S22): Unknown column 'username' in 'where claus
e'
mysql> select * from credential where Name = 'admin' limit 10
;
+----+-------+-------+--------+-------+----------+-----------
--+---------+-------+----------+----------------------------
-------------+
| ID | Name  | EID   | Salary | birth | SSN      | PhoneNumbe
r | Address | Email | NickName | Password
             |
+----+-------+-------+--------+-------+----------+-----------
--+---------+-------+----------+----------------------------
-------------+
|  6 | Admin | 99999 | 400000 | 3/5   | 43254314 |
  |         |       |        | a5bdf35a1df4ea895905f6f6618e
83951a6effc0 |
+----+-------+-------+--------+-------+----------+-----------
```

Modification of Data Or Updating of data via SQL Query:

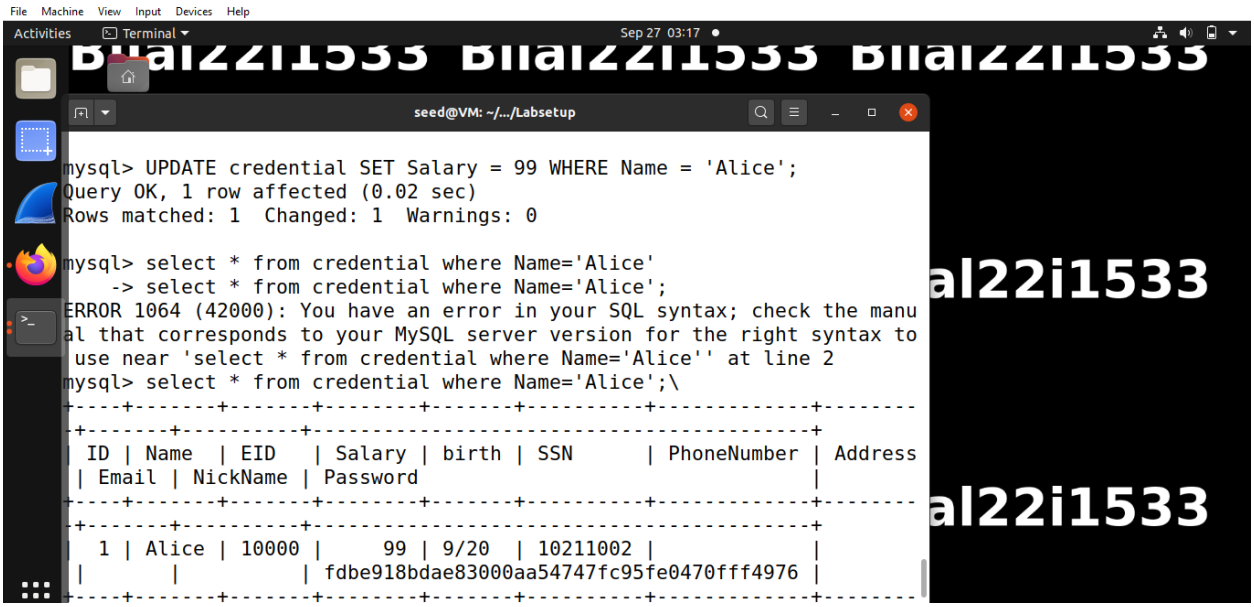
• I updated the `credentials` table to change the admin's salary to `99`.

• The update targeted the admin row (Name = `admin`) and modified the `Salary` field.

• This is a destructive action — it changes stored data permanently in the database.

• Performed only in the SEED lab (controlled environment) where such tests are allowed.

• Evidence to include: full-screen screenshot showing the `credentials` table after the update (showing admin's Salary = 99) with VM wallpaper (name & regno), username, date and time visible.



Demonstration of prevention:

Prevention Proof:

• After modifying `unsafe_home.php` to use prepared statements and proper input handling, I verified the fix through the browser.

• I attempted to log in with a typical SQL injection payload (`admin'#`) that previously bypassed authentication.

• This time, the injection did not work — the application treated the entire string as plain input and failed authentication.

• The test confirmed that the SQL query logic is no longer altered by malicious input.

• This demonstrates that the implemented changes successfully prevented SQL injection through this attack vector.

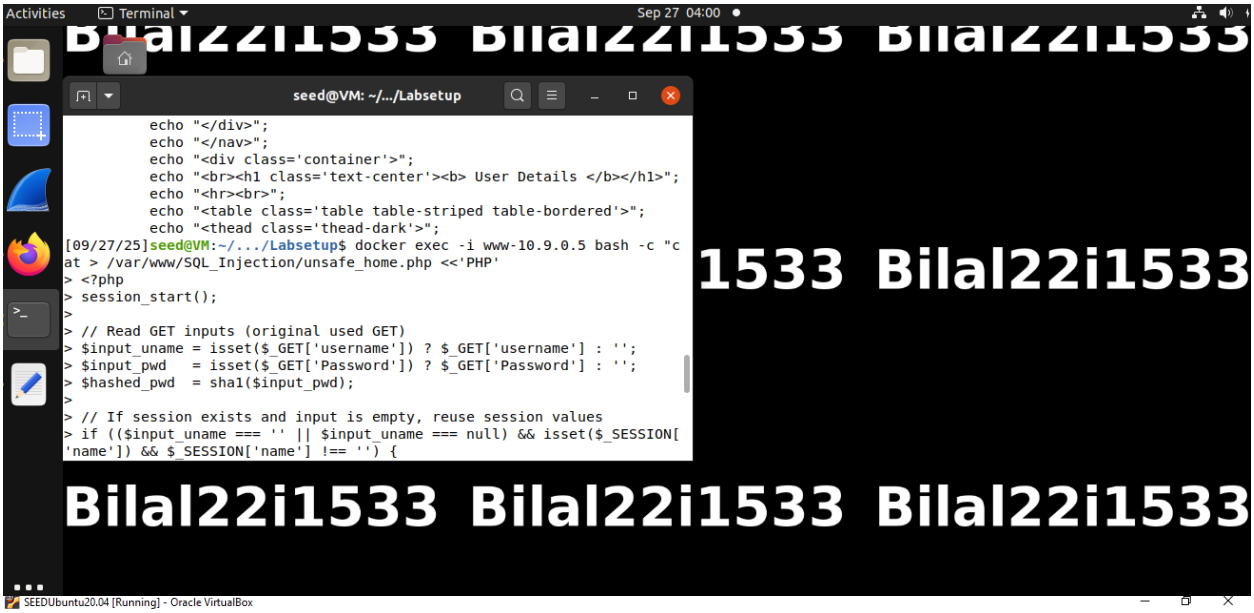
Steps:

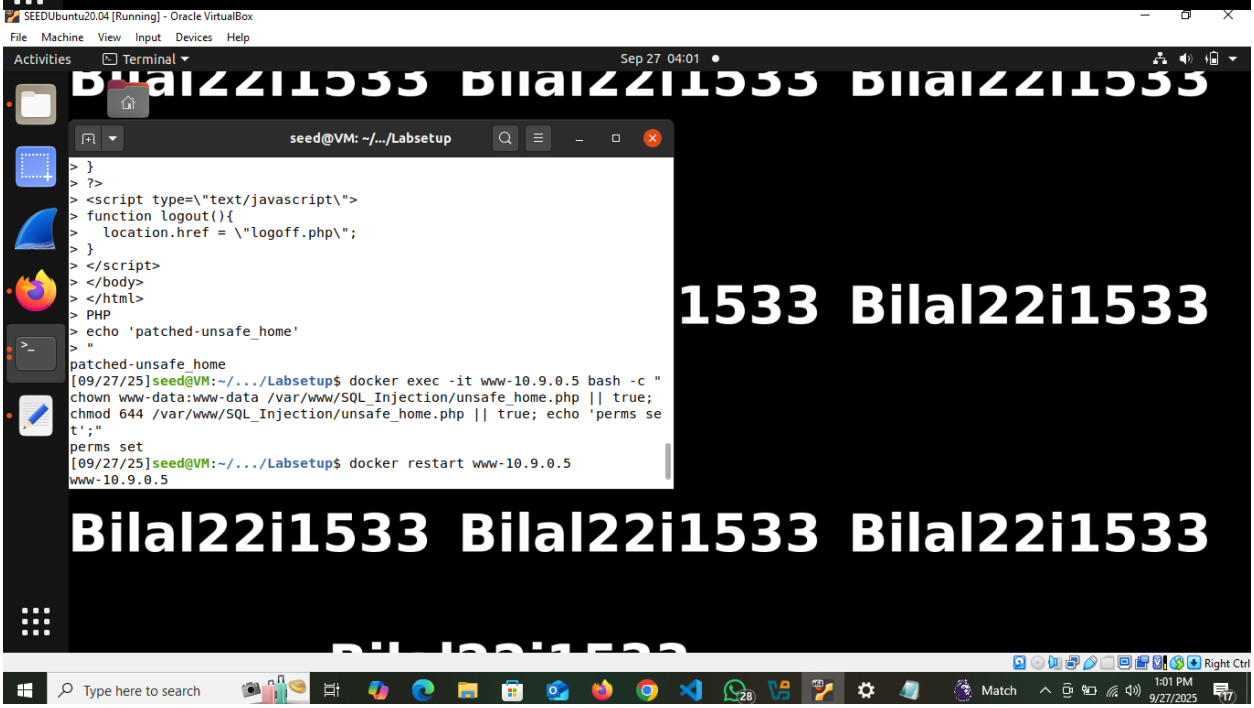
• **Replaced string-concatenated SQL with a prepared statement** — stopped building SQL by joining `$_POST` values into the query.

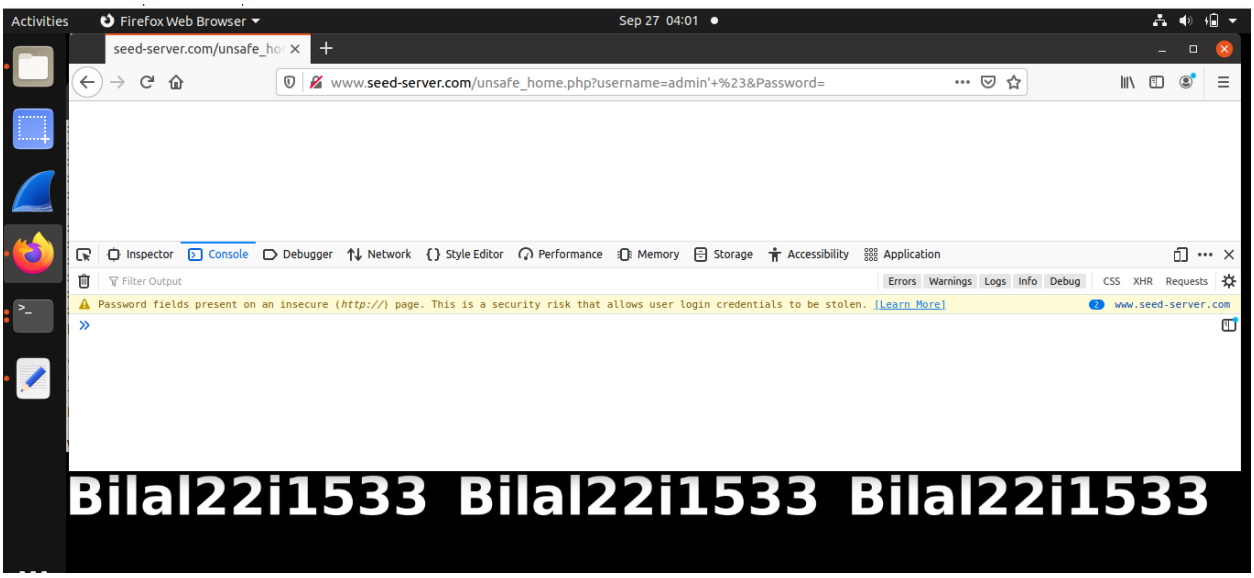
• **Bound user inputs to parameters** (e.g., `bind_param`) so inputs are treated as data, not SQL.

• **Normalized and validated inputs**: `trim()` the values, enforce a maximum length for `Name`, and cast `Salary` to an integer before use.

• **Removed any code that accepted raw SQL fragments** from user input (no `$_POST` inserted directly into SQL).


Activities
Terminal
Sep 27 04:00
seed@VM: ~/.../Labsetup
echo " ";
echo "</nav>";
echo " ";
echo "
<h1 class='text-center'> User Details </h1>";
echo "<hr>
";
echo "<table class='table table-striped table-bordered'>";
echo "<thead class='thead-dark'>";
[09/27/25]seed@VM:~/.../Labsetup$ docker exec -i www-10.9.0.5 bash -c "c
at > /var/www/SQL_Injection/unsafe_home.php <<'PHP'
> <?php
> session_start();
>
> // Read GET inputs (original used GET)
> $input_uname = isset($_GET['username']) ? $_GET['username'] : '';
> $input_pwd   = isset($_GET['Password']) ? $_GET['Password'] : '';
> $hashed_pwd = sha1($input_pwd);
>
> // If session exists and input is empty, reuse session values
> if (($input_uname === '' || $input_uname === null) && isset($_SESSION[
'name']) && $_SESSION['name'] !== '') {
Bilal22i1533 Bilal22i1533 Bilal22i1533


SEEDUbuntu20.04 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities
Terminal
Sep 27 04:01
seed@VM: ~/.../Labsetup
> }
> ?>
> <script type=\"text/javascript\">
> function logout(){
> location.href = \"logoff.php\";
> }
> </script>
> </body>
> </html>
> PHP
> echo 'patched-unsafe_home'
> "
patched-unsafe_home
[09/27/25]seed@VM:~/.../Labsetup$ docker exec -it www-10.9.0.5 bash -c "
chown www-data:www-data /var/www/SQL_Injection/unsafe_home.php || true;
chmod 644 /var/www/SQL_Injection/unsafe_home.php || true; echo 'perms se
t';"
perms set
[09/27/25]seed@VM:~/.../Labsetup$ docker restart www-10.9.0.5
www-10.9.0.5
Bilal22i1533 Bilal22i1533 Bilal22i1533
Type here to search
Match
1:01 PM
9/27/2025



What I learned from Lab 1 (Reflection):

- **How SQL injection works**: I learned how concatenating unsanitized user input into SQL queries allows an attacker to alter the intended SQL logic (e.g., always-true conditions, stacked queries, UNION selects).
- **Common techniques**: Authentication bypass (`' OR '1'='1' --`), union-based data extraction, and blind SQL injection for retrieving data when responses are not directly shown.

- **Impact**: SQLi can leak sensitive information (passwords, PII), allow unauthorized actions (data modification or deletion), and compromise the backend database.
- **Why fixes work**: Parameterized queries separate code from data so injected tokens are treated as data (not SQL). Input validation and least-privilege reduce the attack surface.
- **Operational considerations**: Logging, monitoring, and regular code reviews are essential; security-by-default (ORMs, prepared statements) prevents many issues.
- **Practical skills gained**: Using the VM, installing Docker, running and interacting with a vulnerable web app, crafting payloads, and applying fixes in code.